

Sistema de Estacionamento

Keven Pereira dos Santos, Caio Alves da Silva

2 de julho de 2026

Resumo

Este relatório apresenta o desenvolvimento do EasyParking, um sistema web para gerenciamento de estacionamentos, desenvolvido com o objetivo de automatizar o controle de reservas, estadias, veículos, multas, reclamações e geração de relatórios. O sistema possui três perfis de acesso (cliente, funcionário e gerente), cada um com funcionalidades específicas, garantindo segurança e controle de permissões. Ao longo do documento são apresentados os requisitos funcionais e não funcionais, os diagramas UML utilizados na modelagem, a documentação dos principais endpoints da API e os testes realizados para validação das funcionalidades implementadas. Os resultados obtidos demonstram que a maior parte das funcionalidades foi implementada com sucesso, embora algumas divergências tenham sido identificadas durante os testes, indicando oportunidades de melhoria para versões futuras do sistema.

1 Objetivos do Sistema

- Realizar login e cadastro de clientes e funcionários
- Gerenciar Pátios e disponibilidade de vagas
- Adicionar e Remover reservas
- Gerenciar veículo e respectivos motoristas
- Aplicar Multas

2 Equipe e Funções

1. Keven Pereira dos Santos: responsável pelo BackEnd e Diagramas
2. Caio Alves da Silva: responsável pelo FrontEnd e Testes de Funcionalidades do Sistema

3 Requisitos

3.1 Requisitos Funcionais

- RF01 – Autenticação de usuários

O sistema deve permitir o login de clientes, funcionários e gerentes por meio de credenciais válidas.

- RF02 – Cadastro de clientes
O sistema deve permitir que novos clientes realizem seu cadastro.
- RF03 – Gerenciamento do perfil
O sistema deve permitir que clientes, funcionários e gerentes visualizem e atualizem seus próprios dados cadastrais.
- RF04 – Gerenciamento de veículos
O sistema deve permitir que o cliente cadastre, edite e remova seus veículos.
- RF05 – Gerenciamento de motoristas autorizados
O sistema deve permitir que o cliente adicione e remova motoristas autorizados para dirigir cada veículo cadastrado.
- RF06 – Gerenciamento de reservas
O sistema deve permitir que o cliente crie e cancele reservas de vagas em um pátio.
- RF07 – Consulta de reservas
O sistema deve permitir que funcionários visualizem as reservas cadastradas.
- RF08 – Controle de entrada
O sistema deve permitir que o funcionário valide a entrada dos veículos conforme a reserva realizada.
- RF09 – Controle de saída
O sistema deve permitir que o funcionário registre a saída dos veículos.
- RF10 – Aplicação de multas
O sistema deve permitir que o funcionário aplique multas aos clientes quando necessário.
- RF11 – Gerenciamento de clientes
O sistema deve permitir que o funcionário pesquise, visualize e atualize os dados cadastrais dos clientes.
- RF12 – Registro de reclamações
O sistema deve permitir que o cliente registre reclamações relacionadas às suas reservas ou estadias.
- RF13 – Gerenciamento de reclamações
O sistema deve permitir que o gerente visualize todas as reclamações e altere o status de cada uma.
- 14 – Gerenciamento de pátios
O sistema deve permitir que o gerente cadastre, edite e remova pátios de estacionamento.
- 15 – Geração de relatórios
O sistema deve permitir que o gerente gere relatórios mensais contendo informações operacionais do estacionamento.
- RF16 – Download de relatórios
O sistema deve permitir que os relatórios mensais sejam exportados e baixados em formato PDF.

3.2 Requisitos não funcionais

- RNF01 – Segurança
O sistema deve permitir acesso às funcionalidades somente após autenticação do usuário.
- RNF02 – Controle de permissões
Cada tipo de usuário (cliente, funcionário e gerente) deve possuir acesso apenas às funcionalidades correspondentes ao seu perfil.
- RNF03 – Persistência de dados
Todas as informações cadastradas devem ser armazenadas em banco de dados relacional de forma consistente.
- RNF04 – Integridade dos dados
O sistema deve garantir a integridade dos dados, impedindo operações que violem os relacionamentos entre as entidades.
- RNF05 – Desempenho
As operações de consulta e cadastro devem apresentar tempo de resposta adequado para utilização em ambiente web.
- RNF06 – Usabilidade
O sistema deve possuir interface intuitiva e de fácil utilização para todos os perfis de usuários.
- RNF07 – Compatibilidade
O sistema deve ser acessível por navegadores web modernos, mantendo funcionamento adequado independentemente do navegador utilizado.
- RNF08 – Disponibilidade
O sistema deve permanecer disponível para utilização durante seu período de funcionamento, salvo interrupções para manutenção.
- RNF09 – Confiabilidade
As operações de cadastro, atualização e remoção devem ser executadas de forma consistente, evitando perda de informações.
- RNF10 – Exportação de dados
Os relatórios gerados pelo sistema devem poder ser exportados em formato PDF para armazenamento no dispositivo do usuário.

4 Diagramas

Esta seção contém toda a parte de UML do projeto, desde os casos de uso até os diagramas de sequência. Devido ao compilamento do arquivo, as imagens foram comprimidas. Caso a visualização do diagrama esteja prejudicada, as imagens originais podem ser encontradas no repositório do projeto:

4.1 Casos de Uso

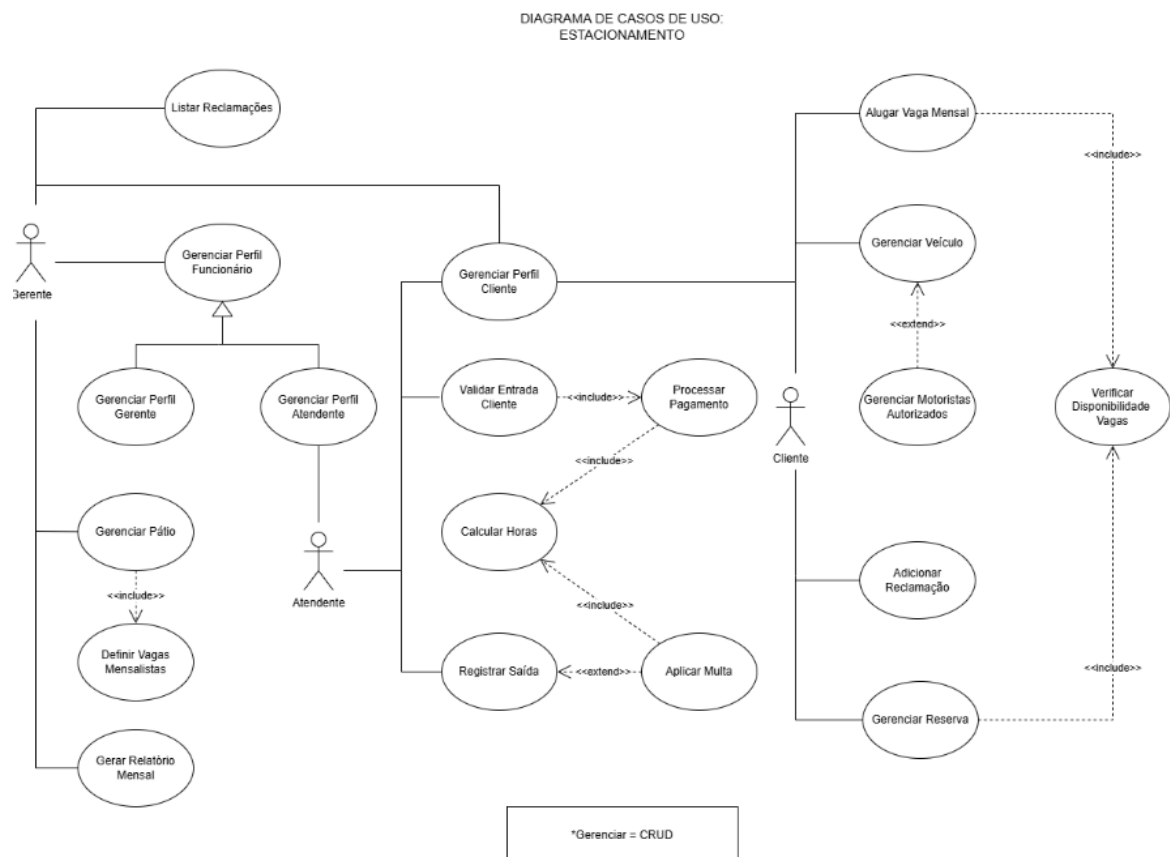


Figura 1: Casos de Uso.

4.2 Diagrama de Classes

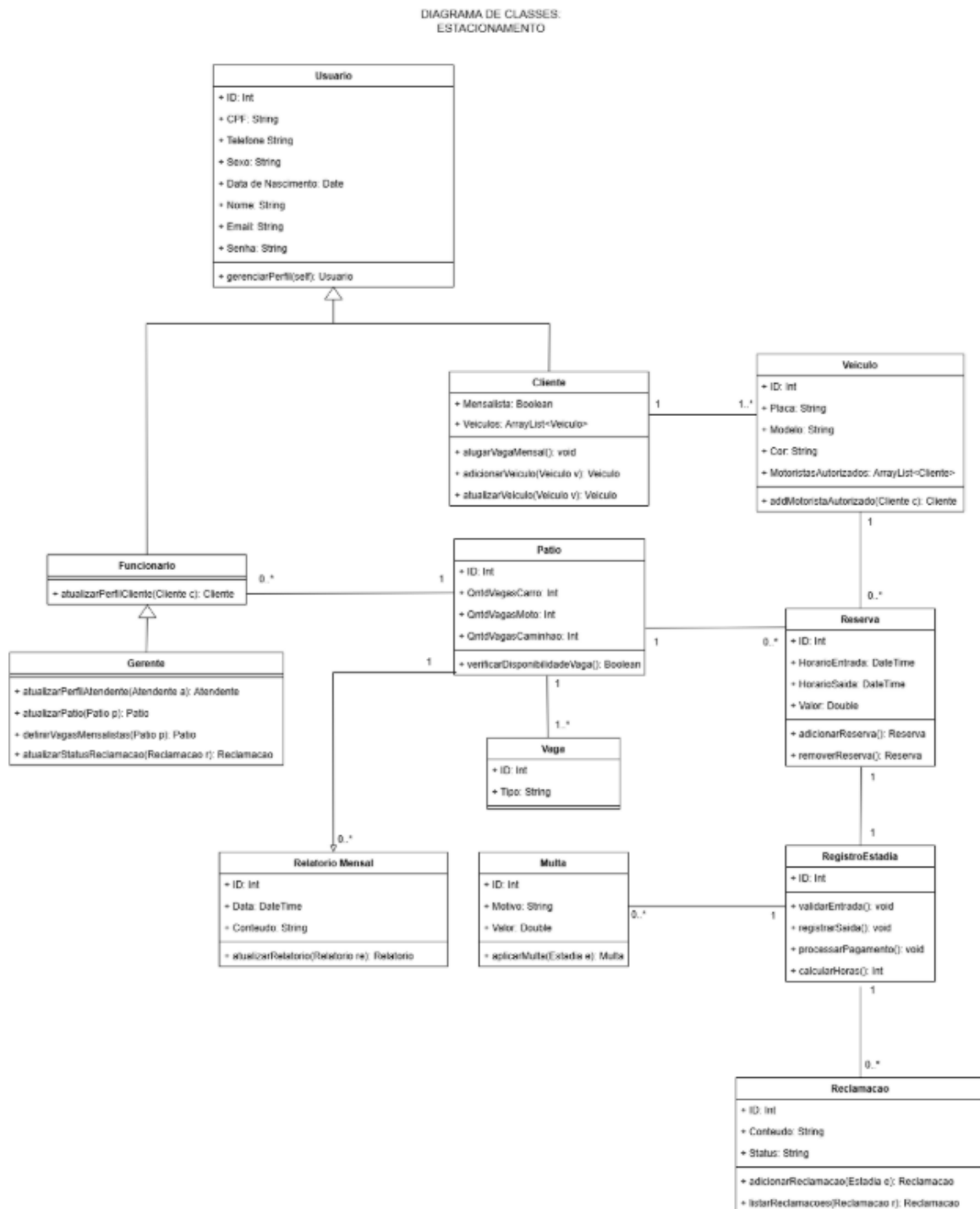


Figura 2: Diagrama de Classes.

4.3 Diagramas Entidade-Relacionamento

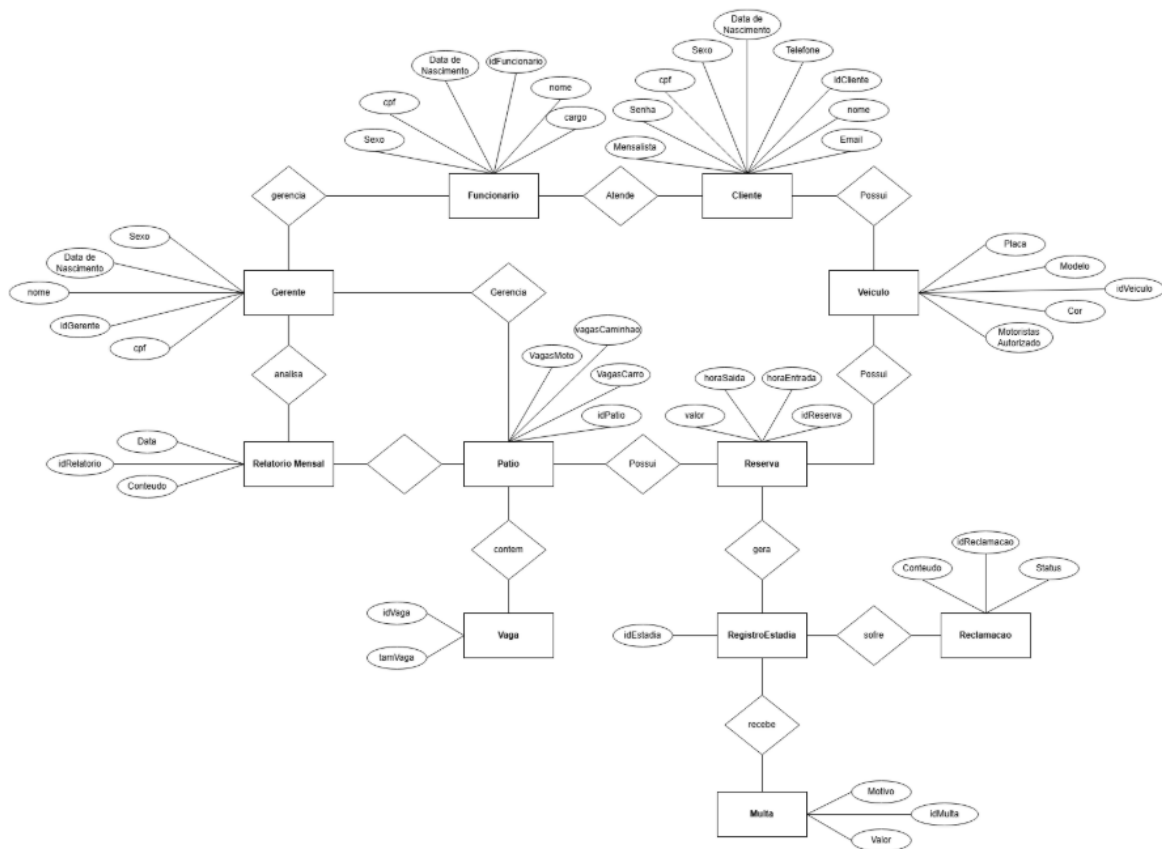


Figura 3: Diagrama Entidade-Relacionamento.

4.4 Diagramas de Sequência

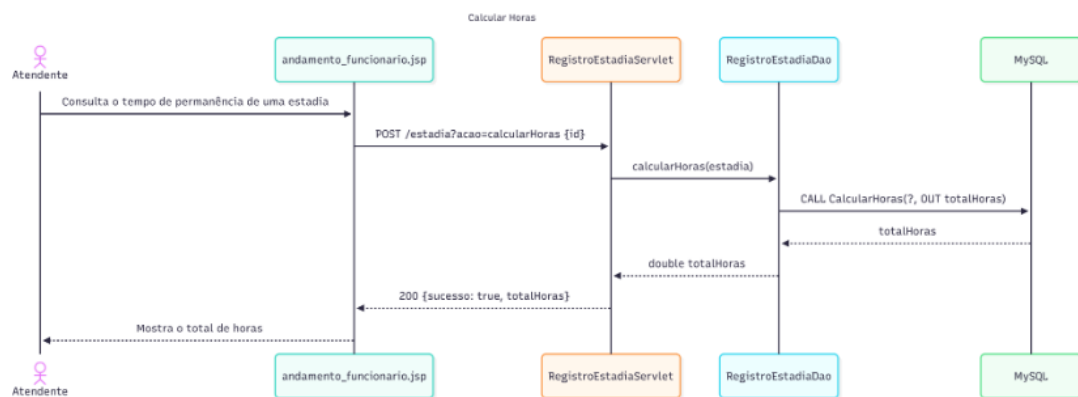


Figura 4: Caso de Uso: Calcular Total de Horas.

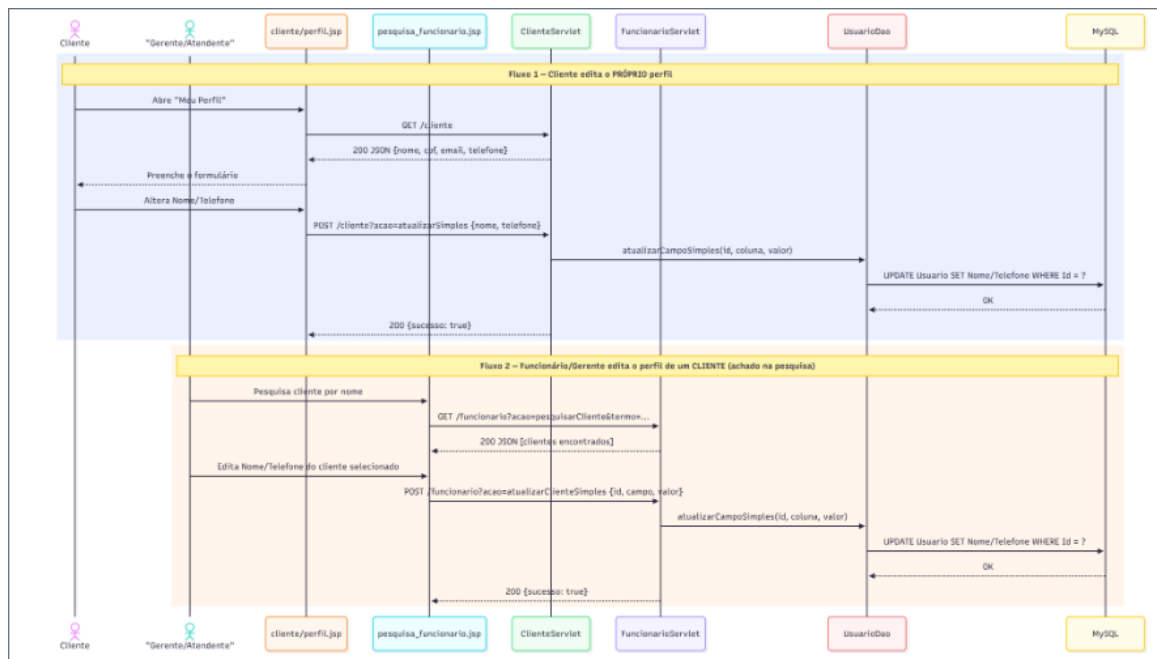


Figura 5: Caso de Uso: Editar Perfil de Cliente.



Figura 6: Caso de Uso: Editar Perfil de Gerente.



Figura 7: Caso de Uso: Gerenciar Perfil de Funcionario.

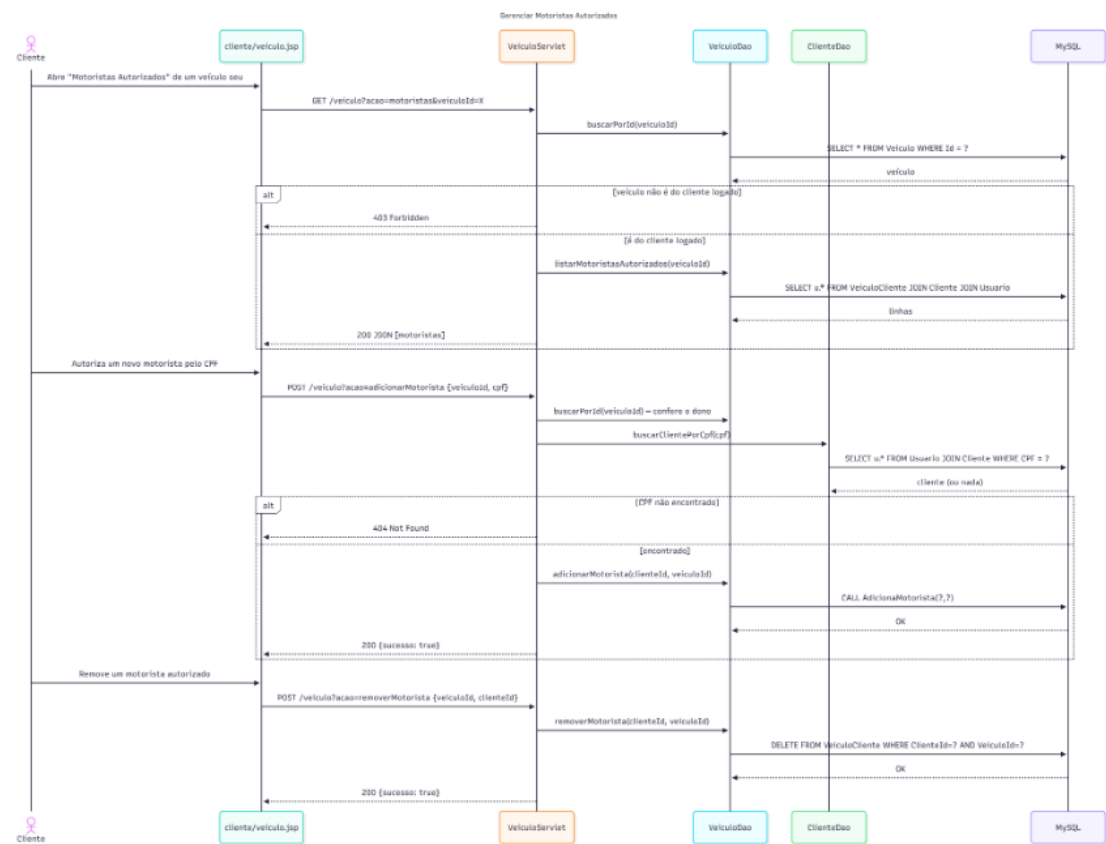


Figura 8: Caso de Uso: Gerenciar Motoristas Autorizados.

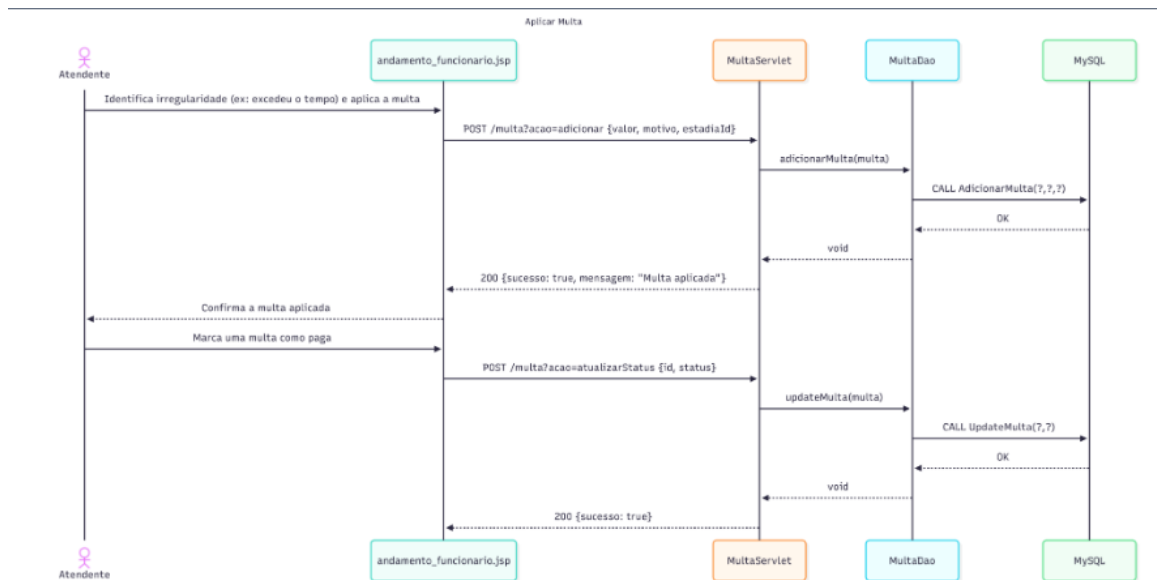


Figura 9: Caso de Uso: Gerenciar Multas.

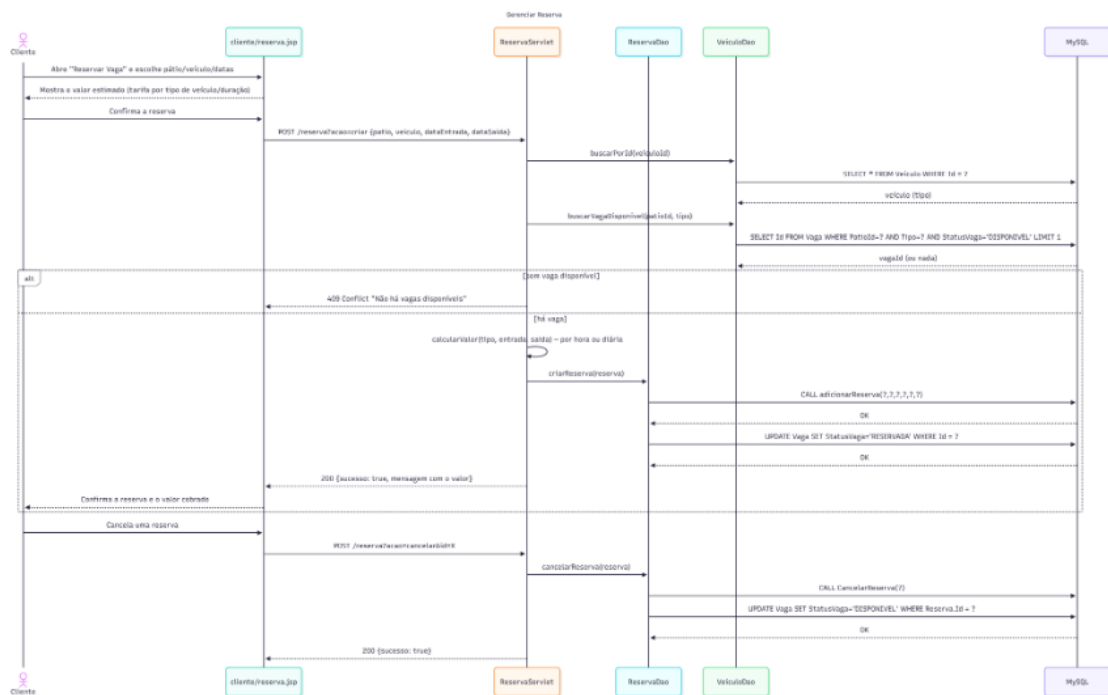


Figura 10: Caso de Uso: Gerenciar Reservas.



Figura 11: Caso de Uso: Gerenciar Veículo.

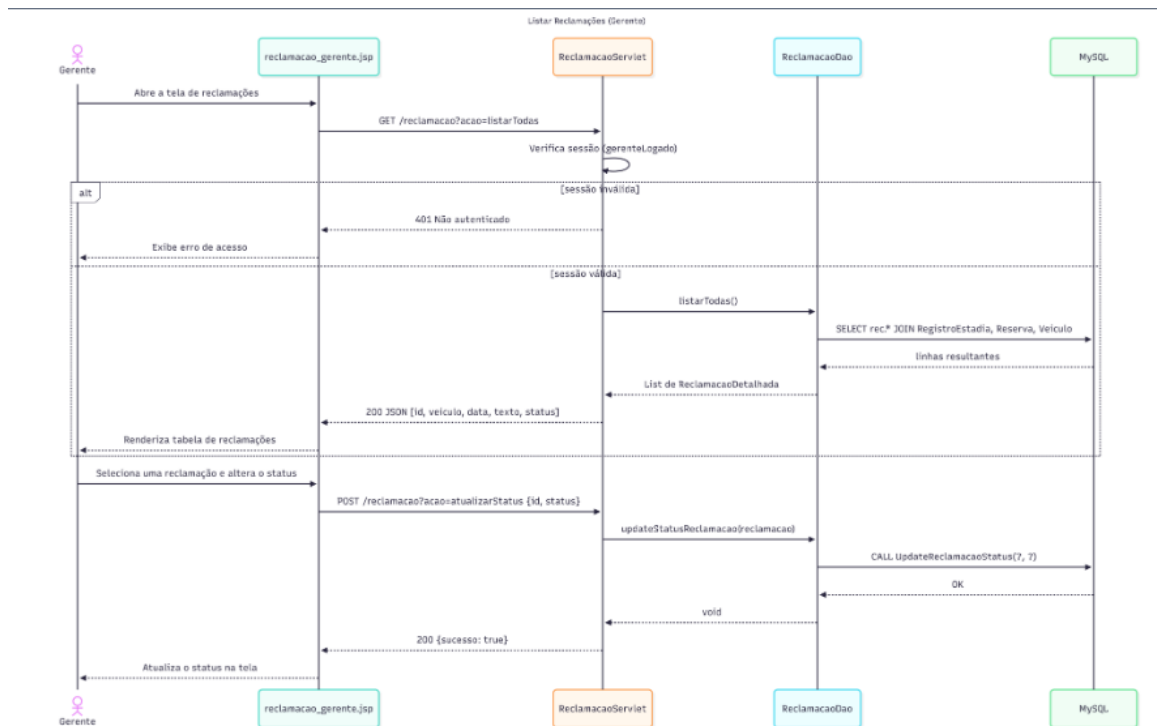


Figura 12: Caso de Uso: Listar Reclamações.

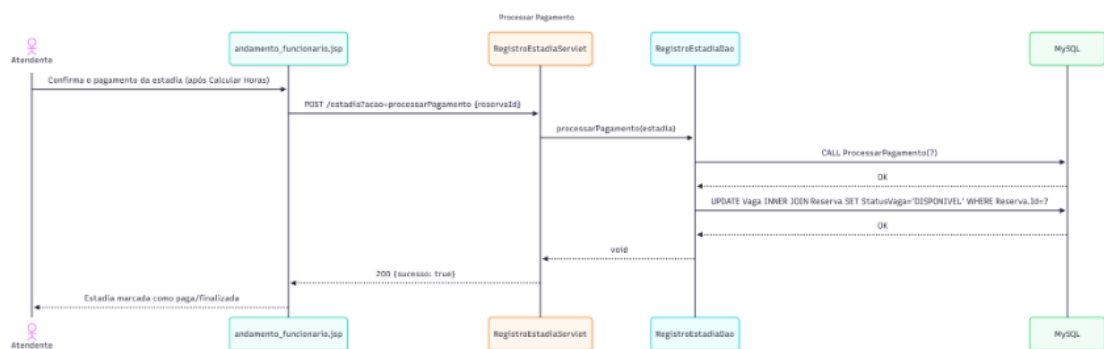


Figura 13: Caso de Uso: Processar Pagamento.

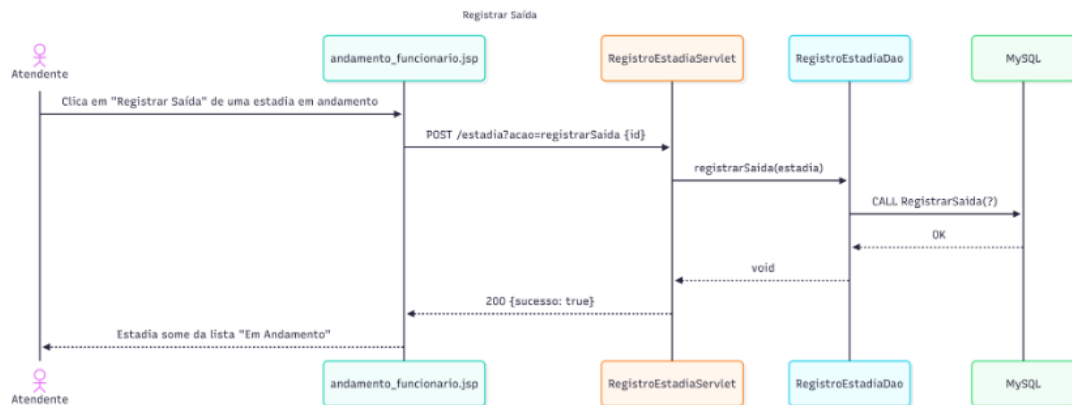


Figura 14: Caso de Uso: Registrar Saída.



Figura 15: Caso de Uso: Relatório Mensal.

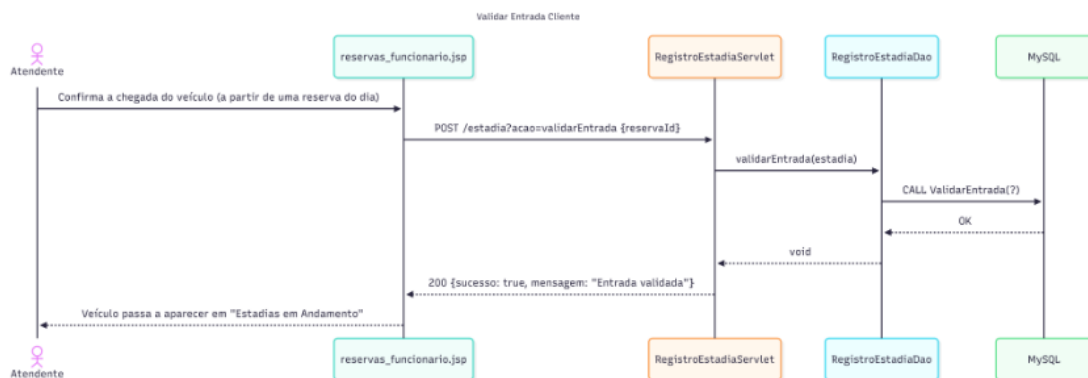


Figura 16: Caso de Uso: Validar Entrada.



Figura 17: Caso de Uso: Verificar Disponibilidade de Vagas.

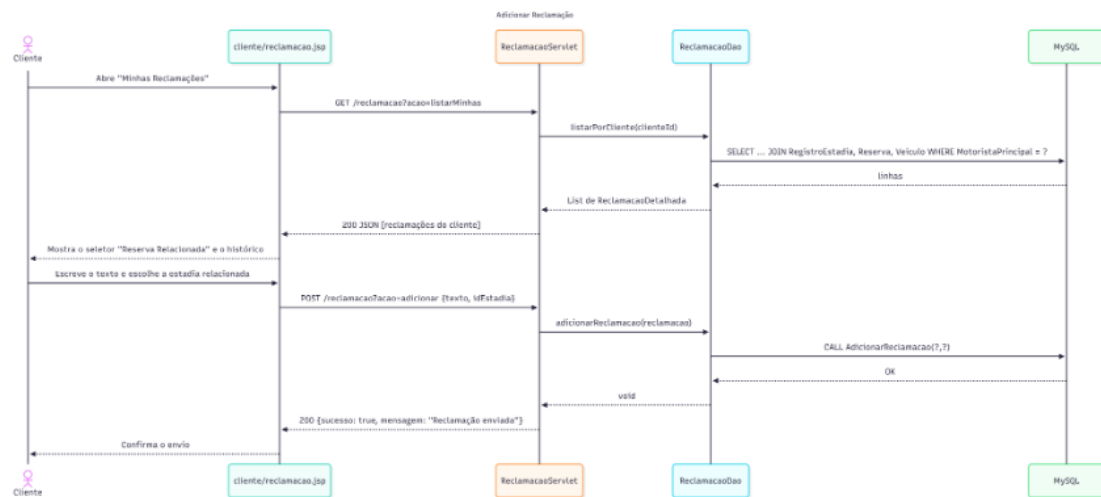


Figura 18: Caso de Uso: Adicionar Reclamação.

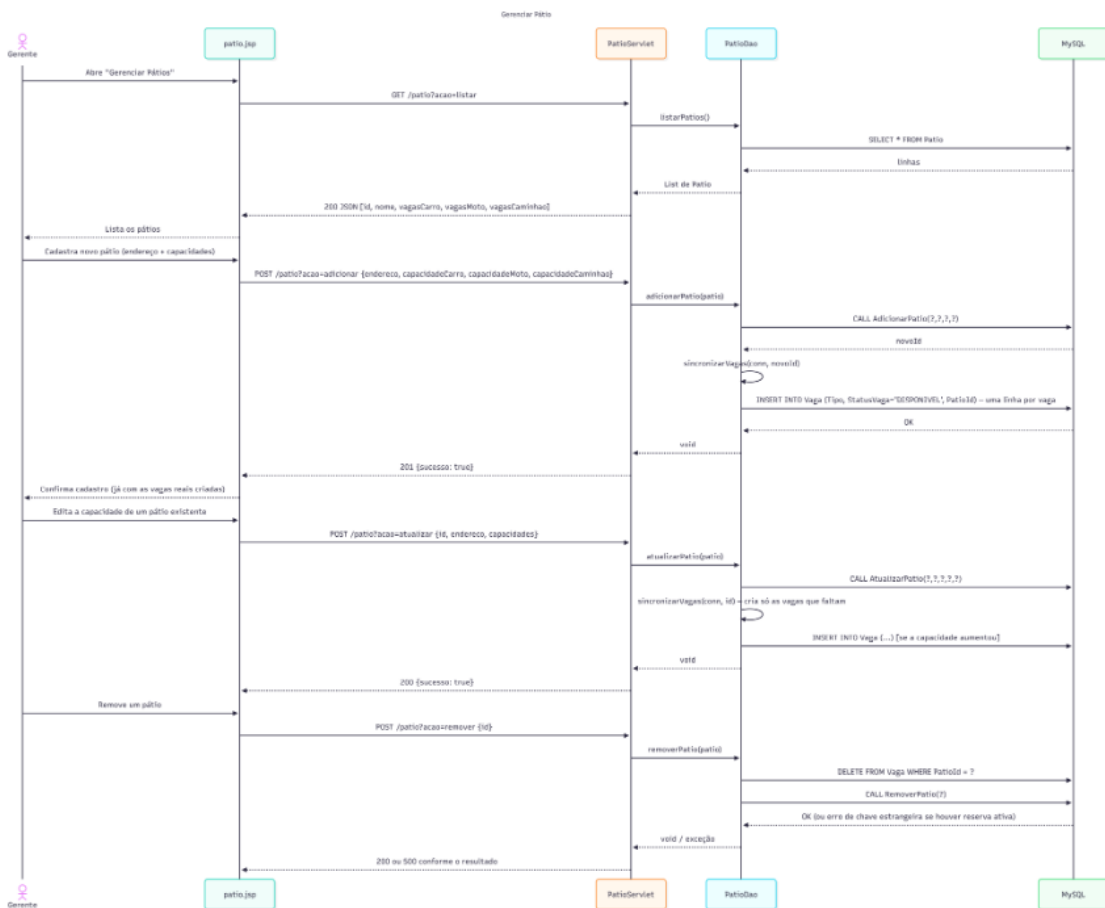


Figura 19: Caso de Uso: Gerenciar Pátio.

5 Testes e Validação

5.1 Relatório de Testes de Endpoints

Esta seção documenta todos os endpoints implementados na API do EasyParking (sistema de gerenciamento de estacionamento), organizados por área funcional. Para cada endpoint são apresentados o método HTTP, os parâmetros de entrada, um exemplo de requisição e a resposta

esperada em formato JSON. A documentação foi extraída diretamente da implementação real dos Servlets (camada *controller*) e validada com uma coleção de testes no Postman, cobrindo autenticação, cadastro e gestão de clientes, funcionários e gerentes, pátios e vagas, veículos, reservas, estadias, reclamações, multas e relatórios mensais.

Observação sobre os métodos HTTP: a API utiliza apenas os métodos GET (para consultas) e POST (para toda operação que cria, atualiza ou remove dados), seguindo um roteamento por parâmetro de ação (?acao=...) dentro de cada Servlet, em vez do padrão REST estrito com PUT e DELETE. Essa escolha de arquitetura foi mantida deliberadamente simples, adequada ao escopo do projeto; não há endpoints que utilizem os verbos PUT ou DELETE.

Observação sobre autenticação: a maioria dos endpoints exige uma sessão ativa (cookie JSESSIONID), obtida ao chamar um dos endpoints de login listados na seção 3.1.1. Sem essa sessão, os endpoints protegidos retornam HTTP 401 com {"sucesso": false, "mensagem": "...não autenticado..."}.

5.1.1 Autenticação

Login unificado (detecta automaticamente se a conta é Cliente, Funcionário ou Gerente) e os fluxos de verificação de e-mail no cadastro e recuperação de senha.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Login unificado POST /login	POST	Corpo (JSON): email (string) senha (string)	{ "email": "funcionario.teste@easyparking.com", "senha": "123456" }	200 OK { "sucesso": true, "redirecionar": "funcionario/reservas_funcionario.jsp" } 401 (credenciais inválidas) { "sucesso": false, "mensagem": "E-mail ou senha incorretos!" }
Login direto do funcionário POST /funcionario?acao=login	POST	Query: acao=login Corpo (JSON): email, senha	{ "email": "funcionario.teste@easyparking.com", "senha": "123456" }	200 OK { "sucesso": true, "mensagem": "Login efetuado!", "redirecionar": "reservas_funcionario.jsp" }
Login direto do gerente POST /gerente?acao=login	POST	Query: acao=login Corpo (JSON): email, senha	{ "email": "gerente.teste@easyparking.com", "senha": "123456" }	200 OK { "sucesso": true, "mensagem": "Login efetuado!", "redirecionar": "reclamacao_gerente.jsp" }
Verificação de cadastro - enviar código POST /verificacao-cadastro	POST	Corpo (JSON): acao="enviar" email (string)	{ "acao": "enviar", "email": "novo.cliente@teste.com" }	200 OK { "sucesso": true, "mensagem": "Código enviado para o e-mail." }

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Verificação de cadastro - verificar código POST /verificacao-cadastro	POST	Corpo (JSON): acao="verificar" codigo (string, 6 dígitos)	{ "acao": "verificar", "codigo": "123456" }	200 OK { "sucesso": true, "mensagem": "E-mail verificado." } 400 (código errado) { "sucesso": false, "mensagem": "Código inválido." }
Recuperar senha - enviar código POST /Recuperar-SenhaServlet	POST	Corpo (JSON): acao="enviar_codigo" email (string)	{ "acao": "enviar_codigo", "email": "cliente.teste@easyparking.com" }	200 OK { "sucesso": true, "mensagem": "Se o e-mail existir, um código foi enviado." }
Recuperar senha - verificar código POST /Recuperar-SenhaServlet	POST	Corpo (JSON): acao="verificar_codigo" codigo (string)	{ "acao": "verificar_codigo", "codigo": "123456" }	200 OK { "sucesso": true, "mensagem": "Código válido." }
Recuperar senha - definir nova senha POST /Recuperar-SenhaServlet	POST	Corpo (JSON): acao="nova_senha" senha (string)	{ "acao": "nova_senha", "senha": "novaSenha123" }	200 OK { "sucesso": true, "mensagem": "Senha redefinida com sucesso!" }

5.1.2 Usuário (cadastro genérico)

CRUD genérico da tabela *Usuario*, sem vínculo com *Cliente/Funcionário/Gerente*.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Cadastrar usuário POST /usuario?acao=cadastrar	POST	Corpo (JSON): cpf, nome, sexo (MASCULINO/FEMININO), dataNascimento (yyyy-MM-dd), email, telefone, senha	{ "cpf": "11111111111", "nome": "Teste Genérico", "sexo": "MASCULINO", "dataNascimento": "2000-01-01", "email": "generico@teste.com", "telefone": "(11) 99999-0000", "senha": "123456" }	200 OK { "sucesso": true, "mensagem": "Usuário cadastrado com sucesso!" }

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Atualizar usuário POST /usuario?acao=atualizar	POST	Corpo (JSON): id + mesmos campos do cadastro	{ "id": "1", "cpf": "11111111111", "nome": "Teste Atualizado", "sexo": "MASCULINO", "dataNascimento": "2000-01-01", "email": "generico@teste.com", "telefone": "(11) 99999-0000", "senha": "123456" }	200 OK { "sucesso": true, "mensagem": "Dados do usuário atualizados!" }
Remover usuário POST /usuario?acao=remover	POST	Corpo (JSON): id (int)	{ "id": "1" }	200 OK { "sucesso": true, "mensagem": "Usuário removido com sucesso!" }

5.1.3 Cliente

Cadastro público de clientes e gestão do perfil do cliente autenticado (sessão usuarioLogado).

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Cadastrar cliente POST /cliente?acao=cadastrar	POST	Corpo (JSON): cpf, nome, email, telefone, senha, sexo, dataNascimento, mensalista (boolean)	{ "cpf": "98765432100", "nome": "Cliente Teste", "email": "cliente.teste@easyparking.com", "telefone": "(11) 98888-0000", "senha": "123456", "sexo": "FEMININO", "dataNascimento": "1998-03-15", "mensalista": "false" }	201 Created { "sucesso": true, "mensagem": "Cliente cadastrado com sucesso!" } 400 (e-mail/CPF já existe) { "sucesso": false, "mensagem": "Erro ao salvar no banco (e-mail/CPF já cadastrado?)." }
Ver meu perfil GET /cliente	GET	Nenhum (usa a sessão do cliente logado)	(sem corpo)	200 OK { "nome": "Cliente Teste", "cpf": "98765432100", "email": "cliente.teste@easyparking.com", "telefone": "(11) 98888-0000" } 401 (sem sessão) { "sucesso": false, "mensagem": "Usuário não autenticado." }
Atualizar Nome/Telefone POST /cliente?acao=atualizarSimples	POST	Corpo (JSON): nome, telefone	{ "nome": "Cliente Teste Editado", "telefone": "(11) 97777-0000" }	200 OK { "sucesso": true, "mensagem": "Perfil atualizado!" }

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Atualizar Email/Senha POST /cliente?acao=atualizarComplexo	POST	Corpo (JSON): email, senha	{ "email": "cliente.novo@teste.com", "senha": "novaSenha123" }	200 OK { "sucesso": true, "mensagem": "Credenciais alteradas!" }
Virar/sair de mensalista POST /cliente?acao=mudarMensalista	POST	Nenhum (inverte o status atual do cliente logado)	(sem corpo)	200 OK { "sucesso": true, "novoStatus": true }

5.1.4 Funcionário

Pesquisa e edição de clientes pelo funcionário, e gestão do próprio perfil (sessão funcionario-Logado).

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Pesquisar cliente por nome GET /funcionario?acao=pesquisarCliente	GET	Query: acao=pesquisarCliente, termo (string)	GET /funcionario?acao=pesquisarCliente&termo=Maria	200 OK { "id": 3, "nome": "Maria Silva", "cpf": "98765432100", "email": "maria@teste.com", "numero": "(11) 98888-0000" }
Ver meu perfil GET /funcionario?acao=buscarPerfil	GET	Query: acao=buscarPerfil	(sem corpo)	200 OK { "nome": "Funcionario Teste", "cpf": "11122233344", "email": "funcionario.teste@easyparking.com", "telefone": "(11) 90000-0001" }
Atualizar Nome/Telefone (próprio) POST /funcionario?acao=atualizarSimples	POST	Corpo (JSON): campo ("nome" ou "numero"), valor	{ "campo": "nome", "valor": "Novo Nome" }	200 OK { "sucesso": true, "mensagem": "Dados atualizados com sucesso!" }
Atualizar Email/Senha (próprio) POST /funcionario?acao=atualizarComplexo	POST	Corpo (JSON): tipo ("email" ou "senha"), valorAtual, novoValor	{ "tipo": "senha", "valorAtual": "123456", "novoValor": "novaSenha123" }	200 OK { "sucesso": true, "mensagem": "Credenciais alteradas com sucesso!" } 400 (valor atual não confere)
Editar Nome/Telefone de um cliente POST /funcionario?acao=atualizarClienteSimples	POST	Corpo (JSON): id (do cliente), campo, valor	{ "id": "3", "campo": "numero", "valor": "(11) 96666-0000" }	200 OK { "sucesso": true, "mensagem": "Dados do cliente atualizados!" }

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Editar Email/Senha de um cliente POST /funcionario?acao=atualizarClienteComplexo	POST	Corpo (JSON): id (do cliente), tipo, novoValor	{ "id": "3", "tipo": "email", "novoValor": "cliente.novo@teste.com" }	200 OK { "sucesso": true, "mensagem": "Credenciais do cliente atualizadas!" }

5.1.5 Gerente

Gestão do próprio perfil do gerente (sessão gerenteLogado).

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Ver meu perfil GET /gerente?acao=buscarPerfil	GET	Query: acao=buscarPerfil	(sem corpo)	200 OK { "nome": "Gerente Teste", "cpf": "55566677788", "email": "gerente.teste@easyparking.com", "telefone": "(11) 90000-0002" }
Atualizar Nome/Telefone POST /gerente?acao=atualizarSimple	POST	Corpo (JSON): campo ("nome" ou "numero"), valor	{ "campo": "telefone", "valor": "(11) 95555-0000" }	200 OK { "sucesso": true, "mensagem": "Perfil do gerente atualizado!" }
Atualizar Email/Senha POST /gerente?acao=atualizarComplexo	POST	Corpo (JSON): tipo ("email" ou "senha"), valorAtual, novoValor	{ "tipo": "email", "valorAtual": "gerente.teste@easyparking.com", "novoValor": "gerente.novo@teste.com" }	200 OK { "sucesso": true, "mensagem": "Credenciais alteradas!" }

5.1.6 Pátio

CRUD de pátios e verificação de disponibilidade de vagas (usada antes de toda reserva).

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar pátios GET /patio?acao=listar	GET	Query: acao=listar	(sem corpo)	200 OK [{ "id": 1, "nome": "Rua Teste, 123", "vagasCarro": 50, "vagasMoto": 20, "vagasCaminhao": 5 }]

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Verificar disponibilidade de vaga GET /pátio?acao=verificarDisponibilidade	GET	Query: acao=verificarDisponibilidade, id (pátio), tipoVeiculo (CARRO/MOTO/CAMINHÃO)	GET /pátio?acao=verificarDisponibilidade&id=1&tipoVeiculo=CARRO	200 OK { "sucesso": true, "vagasDisponiveis": 12 }
Cadastrar pátio POST /pátio?acao=adicionar	POST	Corpo (JSON): endereço, capacidadeCarro, capacidadeMoto, capacidadeCaminhão	{ "endereço": "Rua Teste, 123", "capacidadeCarro": "50", "capacidadeMoto": "20", "capacidadeCaminhão": "5" }	201 Created { "sucesso": true, "mensagem": "Pátio cadastrado com sucesso!" }
Atualizar pátio POST /pátio?acao=atualizar	POST	Corpo (JSON): id + mesmos campos do cadastro	{ "id": "1", "endereço": "Rua Teste, 123 - Atualizado", "capacidadeCarro": "60", "capacidadeMoto": "20", "capacidadeCaminhão": "5" }	200 OK { "sucesso": true, "mensagem": "Pátio atualizado com sucesso!" }
Remover pátio POST /pátio?acao=remover	POST	Corpo (JSON): id (int)	{ "id": "1" }	200 OK { "sucesso": true, "mensagem": "Pátio removido do sistema!" }

5.1.7 Veículo

CRUD de veículos do cliente e gestão de motoristas autorizados por veículo.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar meus veículos GET /veiculo	GET	Nenhum (usa a sessão do cliente logado)	(sem corpo)	200 OK [{ "id": 1, "placa": "ABC1D23", "modelo": "Onix", "cor": "Prata", "tipoVeiculo": "CARRO" }]
Listar motoristas autorizados GET /veiculo?acao=motoristas	GET	Query: acao=motoristas, veiculoId (int)	GET /veiculo?acao=motoristas&veiculoId=1	200 OK [{ "id": 5, "nome": "João Souza", "cpf": "12312312300", "email": "joao@teste.com", "numero": "(11) 91111-0000" }] 403 (veículo não é seu)

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Cadastrar veículo POST /veiculo	POST	Corpo (JSON): placa, modelo, cor, tipoVeiculo (CARRO/MOTO/CAMINHAO)	{ "placa": "ABC1D23", "modelo": "Onix", "cor": "Prata", "tipoVeiculo": "CARRO" }	200 OK { "sucesso": true, "mensagem": "Veículo cadastrado com sucesso!" }
Atualizar veículo POST /veiculo?acao=atualizar	POST	Corpo (JSON): id, modelo, cor, tipoVeiculo	{ "id": "1", "modelo": "Onix Plus", "cor": "Branco", "tipoVeiculo": "CARRO" }	200 OK { "sucesso": true, "mensagem": "Veículo atualizado com sucesso!" }
Remover veículo POST /veiculo?acao=remover	POST	Corpo (JSON): id (int)	{ "id": "1" }	200 OK { "sucesso": true, "mensagem": "Veículo removido com sucesso!" }
Autorizar motorista (por CPF) POST /veiculo?acao=adicionarMotorista	POST	Corpo (JSON): veiculoId, cpf	{ "veiculoId": "1", "cpf": "98765432100" }	200 OK { "sucesso": true, "mensagem": "Maria Silva foi autorizado a dirigir este veículo!" }
Remover motorista autorizado POST /veiculo?acao=removerMotorista	POST	Corpo (JSON): veiculoId, clienteId	{ "veiculoId": "1", "clienteId": "3" }	200 OK { "sucesso": true, "mensagem": "Motorista removido com sucesso!" }

5.1.8 Reserva

Criação e cancelamento de reservas, com verificação de vaga e cálculo automático de tarifa.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar minhas reservas (cliente) GET /reserva	GET	Nenhum (usa a sessão do cliente logado)	(sem corpo)	200 OK [{ "id": 1, "veiculo": "Veículo ID 1", "patio": "Pátio ID 1", "data": "05/07/2026 08:00", "valor": "R\$ 50.00" }]

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar reservas do dia (funcionário) GET /reserva?acao=listarDoDia	GET	Query: acao=listarDoDia	(sem corpo)	200 OK [{ "id": 1, "veiculo": "Unix (Prata) - ABC1D23", "patio": "Rua Teste, 123", "horaEntrada": "05/07/2026 às 08:00", "horaSaida": "05/07/2026 às 18:00", "valor": "R\$ 50.00" }]
Criar reserva POST /reserva?acao=criar	POST	Corpo (JSON): patio (id), veiculo (id), dataEntrada, dataSaida (formato yyyy-MM-dd HH:mm)	{ "patio": "1", "veiculo": "1", "dataEntrada": "2026-07-05 08:00", "dataSaida": "2026-07-05 18:00" }	200 OK { "sucesso": true, "mensagem": "Reserva realizada com sucesso! Valor: R\$ 50.00" } 409 (sem vaga) { "sucesso": false, "mensagem": "Não há vagas disponíveis nesse pátio para o tipo de veículo selecionado." }
Cancelar reserva POST /reserva?acao=cancelar	POST	Query: acao=cancelar, id (int)	POST /reserva?acao=cancelar&id=1 (sem corpo)	200 OK { "sucesso": true, "mensagem": "Reserva cancelada com sucesso!" }

5.1.9 Estadia (entrada, saída e pagamento)

Validação de entrada e saída do veículo no pátio, cálculo de horas e processamento do pagamento.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar estadias em andamento GET /estadia?acao=listarAndamento	GET	Query: acao=listarAndamento	(sem corpo)	200 OK [{ "id": 1, "veiculo": "Unix (Prata) - ABC1D23", "patio": "Rua Teste, 123", "horaEntrada": "05/07/2026 às 08:00", "horaSaida": "-", "valor": "R\$ 50.00" }]
Listar minhas estadias (cliente) GET /estadia?acao=listarMinhas	GET	Query: acao=listarMinhas	(sem corpo)	200 OK [{ "id": 1, "veiculo": "Unix (ABC1D23)", "data": "05/07/2026" }]

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Validar entrada do veículo POST /estadia?acao=validarEntrada	POST	Corpo (JSON): reservaId (int)	{ "reservaId": "1" }	200 OK { "sucesso": true, "mensagem": "Entrada do veículo validada com sucesso!" }
Calcular horas de permanência POST /estadia?acao=calcularHoras	POST	Corpo (JSON): id (int, Id da ESTADIA, não da reserva)	{ "id": "1" }	200 OK { "sucesso": true, "totalHoras": 5.25 }
Registrar saída do veículo POST /estadia?acao=registrarSaida	POST	Corpo (JSON): id (int, Id da ESTADIA)	{ "id": "1" }	200 OK { "sucesso": true, "mensagem": "Saída registrada com sucesso!" }
Processar pagamento POST /estadia?acao=processarPagamento	POST	Corpo (JSON): reservaId (int)	{ "reservaId": "1" }	200 OK { "sucesso": true, "mensagem": "Pagamento processado com sucesso!" }

5.1.10 Reclamação

Abertura de reclamações pelo cliente e listagem/gestão de status pelo gerente.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar todas (gerente) GET /reclamacao?acao=listarTodas	GET	Query: acao=listarTodas	(sem corpo)	200 OK [{ "id": 1, "veiculo": "Onix (Prata) - ABC1D23", "data": "05/07/2026", "texto": "O veículo estava sujo...", "status": "Em análise" }]
Listar minhas (cliente) GET /reclamacao?acao=listarMinhas	GET	Query: acao=listarMinhas	(sem corpo)	200 OK (mesmo formato da listagem do gerente, filtrado pelo cliente logado)
Adicionar reclamação POST /reclamacao?acao=adicionar	POST	Corpo (JSON): texto (string), idEstadia (int)	{ "texto": "O veículo estava sujo ao ser retirado.", "idEstadia": "1" }	200 OK { "sucesso": true, "mensagem": "Reclamação enviada com sucesso!" }

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Atualizar status POST /reclama-cao?acao=atualizarStatus	POST	Corpo (JSON): id (int), status ("EM_ANALISE" / "RESOLVIDA" / "RECU-SADA")	{ "id": "1", "status": "RESOLVIDA" }	200 OK { "sucesso": true, "mensagem": "Status atualizado com sucesso!" }

5.1.11 Multa

Aplicação de multas em estadias e atualização do status de pagamento.

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Aplicar multa POST /multa?acao=adicionar	POST	Corpo (JSON): valor (float), motivo (string), estadiaId (int)	{ "valor": "50.00", "motivo": "Excedeu o tempo de estadia reservado", "estadiaId": "1" }	200 OK { "sucesso": true, "mensagem": "Multa aplicada com sucesso!" }
Atualizar status da multa POST /multa?acao=atualizarStatus	POST	Corpo (JSON): id (int), status ("NAO_PAGO" / "PAGO" / "RETI-RADO")	{ "id": "1", "status": "PAGO" }	200 OK { "sucesso": true, "mensagem": "Status da multa atualizado!" }

5.1.12 Relatório Mensal

Listagem, geração e exportação em PDF dos relatórios mensais (acesso restrito ao gerente).

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Listar relatórios gerados GET /relatorio	GET	Nenhum (usa a sessão do gerente logado)	(sem corpo)	200 OK [{ "id": 1, "gerado": "01/07/2026 00:00", "ganhos": 3500.0, "carros": 40, "motos": 15, "caminhoes": 3, "tempoMedio": 6.5, "reclamacoes": 2, "multas": 1 }]

continua na próxima página

Endpoint	Método	Parâmetros de Entrada	Exemplo de Requisição	Resposta Esperada (JSON)
Gerar relatório do mês atual POST /relatorio?acao=gerar	POST	Query: acao=gerar	(sem corpo)	201 Created { "sucesso": true, "mensagem": "Relatório do mês gerado com sucesso!" }
Baixar PDF de um relatório GET /relatorio?acao=baixarPdf	GET	Query: acao=baixarPdf, id (int)	GET /relatorio?acao=baixarPdf&id=1	200 OK Content-Type: application/pdf (corpo binário do arquivo PDF, para download)

6 Relatório de Funcionalidades

Esta seção apresenta os resultados reais da execução dos testes funcionais sobre os principais casos de uso do EasyParking, cobrindo os três perfis do sistema (Cliente, Funcionário e Gerente). Os testes foram executados manualmente no ambiente de desenvolvimento (Eclipse JEE, Apache Tomcat e MySQL, rodando localmente sob o contexto /Estacionamento-AP02), seguindo o passo a passo descrito para cada caso de uso, com o resultado obtido comparado ao resultado esperado, definido a partir da lógica real implementada no código.

Do total de 10 casos de uso testados, 4 apresentaram alguma divergência em relação ao esperado, detalhadas em cada teste correspondente e resumidas na tabela final desta seção.

Teste 1 — Login e detecção de perfil (Cliente, Funcionário e Gerente)

Perfil testado: Cliente / Funcionário / Gerente

Pré-condições:

- Servidor Tomcat em execução, projeto publicado no contexto /Estacionamento-AP02.
- Banco de dados MySQL ativo (estacionamento_db).
- Existir pelo menos um Cliente, um Funcionário e um Gerente já cadastrados, com email e senha conhecidos.

Passo a passo da execução:

1. Acessar <http://localhost:8080/Estacionamento-AP02/login.jsp>.
2. Preencher email e senha de um Cliente cadastrado e clicar em "Entrar no EasyParking".
3. Sair (ou abrir uma nova sessão) e repetir o login com as credenciais de um Funcionário.
4. Sair novamente e repetir o login com as credenciais de um Gerente.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O login do Cliente redireciona para cliente/reserva.jsp; o login do Funcionário redireciona para funcionario/reservas_funcionario.jsp; o login do Gerente redireciona para gerente/reclamacao_gerente.jsp. Em cada caso, a sessão correta é criada no servidor (usuarioLogado, funcionarioLogado ou gerenteLogado, respectivamente), já que o LoginServlet verifica primeiro se é Gerente, depois Funcionário, e só por último assume Cliente.	O resultado obtido foi exatamente o esperado: o login de Cliente, Funcionário e Gerente redirecionou corretamente para as telas certas, com a sessão correspondente criada em cada caso.

Erros encontrados: NÃO

Teste 2 — Cadastro de novo cliente

Perfil testado: Visitante (não autenticado)

Pré-condições:

- Estar na tela de login (login.jsp).
- Ter acesso a uma caixa de email real para receber o código de verificação, já que o cadastro depende do EmailService.

Passo a passo da execução:

1. Na tela de login, abrir o fluxo de cadastro.
2. Preencher nome, CPF, email e telefone e avançar.
3. Verificar o código recebido por email e digitá-lo no campo de confirmação.
4. Preencher os dados do veículo (modelo, ano, cor e placa) e avançar.
5. Definir a senha, confirmar a senha e finalizar o cadastro.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema envia os dados por POST para usuario?acao=cadastrar, cria o registro do cliente (e do veículo informado) no banco, e o cadastro finalizado permite login imediato com o email e a senha cadastrados.	O cadastro do cliente foi concluído, mas o veículo informado durante o fluxo não foi cadastrado junto — divergência em relação ao esperado.

Erros encontrados: SIM

Descrição: Ao cadastrar um novo cliente informando os dados do veículo (modelo, ano, cor, placa) na etapa correspondente, o veículo não é salvo no banco de dados junto com o cliente.

Teste 3 — Criar reserva de vaga

Perfil testado: Cliente

Pré-condições:

- Estar logado como Cliente.
- O cliente já possuir pelo menos um veículo cadastrado.
- Existir pelo menos um pátio com vaga DISPONÍVEL do mesmo tipo do veículo escolhido.

Passo a passo da execução:

1. Na tela cliente/reserva.jsp, clicar no botão "+" para criar uma nova reserva.
2. Selecionar o pátio desejado e observar se o sistema mostra a disponibilidade de vagas em tempo real (texto verde/vermelho).
3. Selecionar o veículo.
4. Informar data e hora de entrada e de saída.
5. Confirmar a criação da reserva.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema consulta a disponibilidade via <code>patio?acao=verificarDisponibilidade</code> antes da confirmação, calcula o valor pela regra de cobrança (por hora se a estadia prevista for menor que 12h, por diária se for 12h ou mais, com tarifa conforme o tipo de veículo), grava a reserva via <code>reserva?acao=criar</code> , marca a vaga escolhida como RESERVADA, e a nova reserva passa a aparecer na lista do cliente.	A criação da reserva em si funcionou como esperado (disponibilidade verificada, valor calculado e reserva gravada), mas foi identificado um erro de exibição na lista de reservas do cliente.

Erros encontrados: **SIM**

Descrição: Na lista de reservas do cliente, em vez de exibir o modelo do veículo e o nome do pátio, o sistema mostra o ID numérico de cada um.

Teste 4 — Cancelar reserva

Perfil testado: Cliente

Pré-condições:

- Existir pelo menos uma reserva ativa (pode ser a criada no teste anterior).

Passo a passo da execução:

1. Abrir a reserva criada no teste anterior.
2. Clicar em "Cancelar".
3. Confirmar o cancelamento no modal de confirmação.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema chama reserva?acao=cancelar, a reserva deixa de aparecer como ativa na lista do cliente, e a vaga que estava reservada volta para o status DISPONÍVEL (pode ser conferido tentando reservar a mesma vaga novamente logo em seguida).	O resultado obtido foi exatamente o esperado.

Erros encontrados: **NÃO**

Teste 5 — Abrir reclamação

Perfil testado: Cliente

Pré-condições:

- Estar logado como Cliente.
- Possuir ao menos uma reserva/estadia à qual vincular a reclamação.

Passo a passo da execução:

1. Ir para cliente/reclamacao.jsp.
2. Clicar no botão "+", selecionar a reserva relacionada e escrever o texto da reclamação.
3. Enviar a reclamação.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema grava a reclamação via reclamacao?acao=adicionar com status inicial "Em análise", e ela passa a aparecer tanto na lista de reclamações do cliente quanto na lista de reclamações do gerente.	O resultado obtido foi o esperado.

Erros encontrados: **NÃO**

Teste 6 — Editar perfil — campo simples e campo sensível

Perfil testado: Cliente / Funcionário / Gerente

Pré-condições:

- Estar logado, em qualquer um dos três perfis.

Passo a passo da execução:

1. Na tela de perfil, clicar no ícone de lápis do campo "Nome", alterar o valor e salvar.
2. Clicar no ícone de lápis do campo "Email" (ou "Senha"), informar o valor atual e o novo valor, e salvar.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
A edição de um campo simples (nome, telefone) é salva diretamente via <code>acao=atualizarSimples</code> . A edição de um campo sensível (email, senha) exige a confirmação do valor atual antes de trocar, sendo salva via <code>acao=atualizarComplexo</code> . Em ambos os casos, a alteração é refletida imediatamente na tela e persistida no banco.	Foi identificado um erro: a edição de um campo do perfil é refletida apenas na tela, mas não é persistida no banco de dados.

Erros encontrados: **SIM**

Descrição: Ao alterar uma informação do perfil (por exemplo, o nome) e salvar, o novo valor aparece na tela, mas não é de fato gravado no banco — ao atualizar a página, o campo volta para o valor original.

Teste 7 — Validar entrada e registrar saída (check-in / check-out)

Perfil testado: Funcionário

Pré-condições:

- Estar logado como Funcionário.
- Existir uma reserva confirmada para o veículo que vai ser testado.

Passo a passo da execução:

1. Validar a entrada do veículo referente à reserva (check-in).
2. Registrar a saída do veículo (check-out).
3. Calcular as horas em que o veículo permaneceu estacionado.
4. Processar o pagamento da estadia.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
Cada etapa aciona o endpoint correspondente do <code>RegistroEstadiaServlet</code> (<code>acao=validarEntrada</code> , <code>registrarSaida</code> , <code>calcularHoras</code> , <code>processarPagamento</code>). O horário real de entrada e de saída é gravado no banco, o total de horas é calculado corretamente pela stored procedure a partir da diferença entre os dois horários, e, ao processar o pagamento, a vaga correspondente volta para o status <code>DISPONÍVEL</code> .	O resultado obtido foi o esperado.

Erros encontrados: **NÃO**

Teste 8 — Aplicar multa

Perfil testado: Funcionário / Gerente

Pré-condições:

- Existir uma estadia já registrada no sistema.

Passo a passo da execução:

1. Na tela funcionario/andamento_funcionario.jsp, abrir a estadia em andamento.
2. Clicar em "Aplicar multa", informar o valor e o motivo.
3. Confirmar o lançamento da multa.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema grava a multa via multa?acao=adicionar, vinculada à estadia informada, com status inicial NAO_PAGO.	A multa foi lançada, porém foi identificado um erro no campo de status.

Erros encontrados: **SIM**

Descrição: Ao aplicar uma multa manual, o registro é criado no banco, mas o campo de status fica vazio, em vez de ser preenchido automaticamente com o valor esperado (NAO_PAGO).

Teste 9 — Gerenciar pátio (adicionar, editar e remover)

Perfil testado: Gerente

Pré-condições:

- Estar logado como Gerente.

Passo a passo da execução:

1. Em gerente/patio.jsp, clicar no "+" para adicionar um pátio novo, informando endereço e capacidade de vagas por tipo de veículo.
2. Editar o pátio recém-criado, alterando o endereço ou aumentando a capacidade de algum tipo de vaga.
3. Remover o pátio de teste (certificando-se de que nenhuma vaga dele está vinculada a uma reserva ativa).

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
Ao adicionar, o sistema cria o pátio e sincroniza automaticamente as vagas no banco (uma linha por vaga, já com status DISPONÍVEL). Ao editar, os dados são atualizados e vagas extras são criadas caso a capacidade tenha aumentado. Ao remover, o pátio e suas vagas são apagados do banco, desde que nenhuma vaga esteja vinculada a uma reserva ativa.	O resultado obtido foi o esperado.

Erros encontrados: NÃO

Teste 10 — Atualizar status de reclamação

Perfil testado: Gerente

Pré-condições:

- Estar logado como Gerente.
- Existir pelo menos uma reclamação com status "Em análise" (por exemplo, a criada no teste 5).

Passo a passo da execução:

1. Em gerente/reclamacao_gerente.jsp, abrir a reclamação.
2. Selecionar o novo status ("Resolvido" ou "Não resolvido").
3. Confirmar a atualização.

Resultado esperado vs. obtido:

Resultado Esperado	Resultado Obtido
O sistema chama reclamacao?acao=atualizarStatus, atualizando o status da reclamação no banco. A mudança passa a aparecer tanto na tela do gerente quanto na tela do cliente que abriu a reclamação.	O resultado obtido foi o esperado.

Erros encontrados: NÃO

Resumo Geral dos Testes

#	Caso de Uso	Resultado Geral	Erros
1	Login e detecção de perfil (Cliente, Funcionário e Gerente)	Conforme esperado	0
2	Cadastro de novo cliente	Divergência	1
3	Criar reserva de vaga	Divergência	1
4	Cancelar reserva	Conforme esperado	0

#	Caso de Uso	Resultado Geral	Erros
5	Abrir reclamação	Conforme esperado	0
6	Editar perfil — campo simples e campo sensível	Divergência	1
7	Validar entrada e registrar saída (check-in / check-out)	Conforme esperado	0
8	Aplicar multa	Divergência	1
9	Gerenciar pátio (adicionar, editar e remover)	Conforme esperado	0
10	Atualizar status de reclamação	Conforme esperado	0

Considerações finais: dos 10 casos de uso testados, 6 funcionaram exatamente como esperado e 4 apresentaram divergências que infelizmente não foram corrigidas antes da entrega final do sistema:

- Cadastro de novo cliente
- Criar reserva de vaga
- Editar perfil — campo simples e campo sensível
- Aplicar multa

7 Conclusão

O desenvolvimento do EasyParking permitiu aplicar, na prática, conceitos de engenharia de software, modelagem de sistemas, desenvolvimento web, banco de dados e testes de software. O sistema implementado atende às principais necessidades de um estacionamento, oferecendo funcionalidades para gerenciamento de clientes, veículos, reservas, estadias, multas, reclamações, pátios e geração de relatórios, com diferentes níveis de acesso para clientes, funcionários e gerentes.

Os testes realizados demonstraram que a maioria das funcionalidades opera conforme o esperado. Dos dez de dezesseis casos de uso avaliados, seis foram executados com sucesso e quatro apresentaram divergências que não puderam ser corrigidas antes da entrega final do projeto. Apesar dessas limitações, o sistema mostrou-se funcional e consistente para os principais fluxos de utilização.

Como trabalhos futuros, recomenda-se a correção das funcionalidades que apresentaram divergências durante os testes, a ampliação da cobertura de testes automatizados e a implementação de novos recursos que tornem o sistema ainda mais completo e robusto. Dessa forma, o projeto constitui uma base sólida para futuras evoluções e para a aplicação dos conhecimentos adquiridos durante a disciplina.